Short Project Report
Differential-Drive Robot with ESP32-C6 and Future ESP32-S3 Camera Extension

Krzysztof Sawicki, Michal Szerechowicz, Mateusz Skorupinski

June 19, 2026

1 Project Goal

The goal of the project was to build a compact mobile robot that can be controlled wirelessly, move as a differential-drive platform, keep its heading stable using an inertial sensor, and provide a simple browser interface for manual and semi-autonomous operation. The current robot is based on the ESP32-C6, placed on the CodeCell C6 Drive board, which is responsible for driving the robot and running the control logic. An ESP32-S3 camera module was also prepared in software, but it is not currently mounted or used on the robot. It is intended as a future extension that can provide a Wi-Fi access point, live video stream, snapshot endpoint, and camera power/stream controls.

2 Mechanical Design

The robot uses a differential-drive layout. Two DC motors are mounted on the left and right sides of a 3D-printed chassis. By changing the relative speed and direction of these motors the robot can drive forward, reverse, follow a curve, or rotate in place. The chassis files are included in the project as STL models for both a differential-drive and a four-wheel variant. The implemented control software is written for the two-motor differential-drive arrangement.

The main mechanical requirements were compactness, repeatability, and short wiring. The CodeCell C6 Drive board and LiPo battery are mounted centrally to keep the mass close to the robot's centre. This helps both stability and heading control, because the robot rotates more predictably when the mass is not strongly biased to one side.

3 Electronics

The drive robot is built around the CodeCell C6 Drive board. It integrates the ESP32-C6 microcontroller, wireless communication, a BNO085 9-axis IMU, light and proximity sensing, a LiPo/USB-C power path, and dual H-bridge motor drivers. This reduced the amount of custom wiring because the same board handles control, sensing, communication, battery readings, and motor output.

4 ESP32-C6 Drive Firmware

The C6 firmware is an Arduino/PlatformIO project. It uses the CodeCell library and separates the robot behaviour into three FreeRTOS tasks:

- **MotorTask** runs the low-level H-bridge output and applies the latest signed motor commands to Drive1 and Drive2.

Module	Role in the project
ESP32-C6 / CodeCell C6 Drive	Main robot controller and motor driver
BNO085 IMU	Yaw/heading measurement for stabilised movement
Dual H-bridge drivers	Independent control of left and right DC motors
LiPo power path	Battery supply and voltage/level reporting
ESP32-S3 camera board	Future camera extension; not currently used on robot

- **ControlTask** reads the active command, runs the motion logic, updates the IMU, computes motor output, and publishes robot status.
- **CommunicationTask** starts the robot's Wi-Fi access point and serves the REST API used by the browser controller.

The tasks communicate through shared command, motor, and status buffers. Each buffer is protected by a mutex, which avoids inconsistent reads when one task updates a structure while another task is using it.

Listing 1: Task startup on the ESP32-C6

```
xTaskCreate(motorTask, "MotorTask", 2048, NULL, 3, NULL);
xTaskCreate(controlTask, "ControlTask", 4096, NULL, 2, NULL);
xTaskCreate(communicationTask, "CommunicationTask", 8192, NULL, 1, NULL);
```

5 Motor Mixing and Movement

Manual motion is based on two normalized commands: `move` and `rotation`, both limited to the range from -100 to 100. The controller mixes them into left and right motor values:

$$m_1 = \text{clip}(\texttt{move} + \texttt{rotation}), \qquad m_2 = \text{clip}(\texttt{move} - \texttt{rotation})$$

The sign of each result selects direction and the absolute value selects motor speed. Equal values drive the robot straight. A difference between the motors causes a turn, and opposite signs rotate the robot in place. In software this is implemented by `calculateMotorMix()`, with final output stored as a direction bit and an unsigned PWM-like speed value.

To improve reliability at low speed, the firmware adds a start ramp. Small DC motors often need a short torque boost to overcome static friction. The robot therefore begins with a stronger command and smoothly decays toward the requested speed during the first 500 ms of motion.

6 Heading Stabilisation

The BNO085 IMU provides the robot's yaw angle. The controller compares the measured yaw with a target yaw and uses a PID controller to compute a rotation correction. The angular error is wrapped to the shortest path, so an error near the -180/180 degree boundary is handled correctly.

The active tuning constants in the C6 firmware are:

Parameter	Value
K_p	0.25
K_i	0.02
K_d	0.03
Integral limit	± 120
Maximum correction	± 45

Listing 2: Core yaw correction idea

```
float err = angleError(targetYaw, yaw);
yawIntegral = constrain(yawIntegral + err * dt,
                        -YAW_INTEGRAL_LIMIT,
                         YAW_INTEGRAL_LIMIT);
float correction = YAW_KP * err
                 + YAW_KI * yawIntegral
                 + YAW_KD * derivative;
```

This heading loop is used in both direct and target modes. In direct mode the joystick rotation changes the yaw setpoint over time, so the robot turns relative to its current direction while still using IMU feedback. In target mode the heading loop keeps the robot aligned during autonomous travel.

7 Control Modes

The robot supports two operating modes.

Direct mode is used for real-time driving from the browser joystick. The browser periodically sends `/move?move=...&rotation=...`. If these commands stop arriving for more than 300 ms, the C6 firmware stops the motors. This timeout prevents uncontrolled motion after Wi-Fi or browser failure.

Target mode accepts a distance, relative angle, and optional IMU flag through the `/target` endpoint. The robot first rotates toward the requested heading. When the yaw error is small enough, it drives forward or backward while the PID loop maintains the heading. Distance can be estimated by a calibrated time model or by integrating the linear acceleration from the IMU. The time method is simple and repeatable after calibration, while the IMU method is more experimental because double integration accumulates drift.

8 Wireless API and Web Interface

The C6 robot creates its own Wi-Fi access point named `CodeCell-C6-Drive`, so it does not require an external router. A phone or laptop connects to the access point and opens the single-page controller. The API is intentionally simple and can be tested directly from a browser.

Endpoint	Function
`/move`	Manual movement command
`/target`	Distance and angle command
`/status`	Battery, heading, distance, motor, and PID data
`/estop`	Immediate emergency stop
`/clear_estop`	Clear the emergency stop state

The HTML control app contains a touch-friendly virtual joystick, target-command inputs, emergency buttons, and a live JSON status panel. It sends joystick commands every 100 ms and polls status once per second. Releasing the joystick automatically recentres it and sends a zero-speed command.

9 Future ESP32-S3 Camera Subsystem

The ESP32-S3 project was developed as a possible future camera subsystem. At the current stage it is not installed on the robot and is not part of the active drive platform. The firmware nevertheless configures the camera pins, initializes the sensor in JPEG mode, starts a Wi-Fi access point named `ESP32_S3`, and serves a small camera web page. The camera firmware exposes:

- `/stream` on port 81 for an MJPEG live stream,
- `/capture` on port 80 for a single JPEG snapshot,
- `/control?cmd=...` for stream and camera power commands,
- `/control?cmd=status` for camera/stream state in JSON.

The camera can be turned off in software with `camera_off`, which stops the stream and deinitializes the camera driver. It can then be reinitialized with `camera_on`. If added in the future, this S3 subsystem could be used for monitoring the robot without tying camera management to the real-time drive controller.

10 Safety and Robustness

Several safeguards were implemented. Emergency stop is global and overrides normal control by forcing both motor outputs to zero. The command timeout stops the robot if the browser stops sending fresh direct-control commands. Shared task state is protected by mutexes. Motor output is separated from control calculation, so even when the communication task is busy, the motor task still runs regularly. The browser joystick also recentres on pointer release, which reduces the chance of leaving a nonzero command active accidentally.

11 Summary

The project produced a compact differential-drive robot with a 3D-printed mechanical base, integrated C6 motor-control electronics, IMU-based heading stabilisation, manual joystick driving, target-based movement, a browser REST interface, and emergency handling. The ESP32-S3 camera firmware is prepared as a future extension for live video and snapshots through a separate Wi-Fi camera service. Therefore, the current platform is already usable as a C6-based mobile robot, while future work can add obstacle avoidance, odometry, multi-waypoint navigation, and integration between camera feedback and motion control.